

Automating Hydrogen Legislation Database Consolidation

<https://gemini.google.com/app/ae65facdfd5b90ad>

User prompt: I have a set of lists and documents holding information about hydrogen-related legislation in Czech Republic and Europe. I need to consolidate all the sources into one database that will be the single point of authority for my further research. I want to do that automatically or to automate the whole process as much as I can. This means that I need to: 1. parse the heterogenous data 2. put all the information into one well structured database 3. check all the entries for possible duplicates and hallucinations 4. fill in missing records into the database Can this be accomplished by creating a suitable set of AI agents?

Response: Yes, this can absolutely be accomplished using a **multi-agent document processing pipeline** (often referred to as agentic document extraction). Instead of relying on rigid, rule-based scripts that break when document layouts change, an agentic architecture uses a team of specialized AI models that plan, execute, and validate a multi-step workflow to turn unstructured files into a verified database.

To achieve your goal of creating a single point of authority for Czech and European hydrogen legislation, you will need to assign specific responsibilities to individual agents rather than relying on one massive prompt.

Here is the blueprint for the agents you need and how they interact:

The Agent Roster

Agent Role	Primary Function	Core Technology / Capability
Parser Agent	Normalizes heterogeneous data (PDFs, scans, web pages, Word docs).	Vision-Language Models (VLMs) for layout analysis; interprets tables, columns, and headers visually before reading text.
Extraction Agent	Pulls specific legislative entities (Directives, dates, amendments, jurisdictions) from the text.	Schema-constrained LLM; outputs deterministic, structured data (e.g., JSON) based on your database requirements.
Verification Agent	Prevents hallucinations by independently cross-checking the extracted data against the source document.	Visual grounding; links every extracted field to its exact coordinate or paragraph in the source file for auditing.
Reconciliation Agent	Checks for duplicates and manages versioning (e.g., linking an EU directive to its Czech legislative transposition).	Semantic search and vector embeddings; compares new entries against the existing database to merge or flag conflicts.
Research Agent	Fills in missing data by querying external legal databases.	Tool-use (APIs); capable of searching EUR-Lex or Czech e-Sbírka to find missing enactment dates or related amendments.

The Automated Pipeline

The true power of this system comes from how the agents collaborate in a sequence. Order is critical here—data must be verified before it is reconciled against your database.

- 1

Ingestion & Layout Parsing
Rejects unreadable files automatically

The Parser Agent receives the raw file. Instead of blindly extracting text, it analyzes the document's visual structure. It identifies where the main legislative text is versus side notes, headers, or footnotes, ensuring context isn't lost during text conversion.
- 2

Structured Extraction
Outputs strictly to JSON schema

The Extraction Agent takes the parsed text and identifies the key variables needed for your database (e.g., `legislation_type`, `hydrogen_focus_area`, `jurisdiction`, `status`). It maps these unstructured concepts into a rigid data structure.
- 3

Verification & Anti-Hallucination
Self-correcting retry loop

Before anything is saved, the Verification Agent audits the Extraction Agent's work. If the Extraction Agent claims a law goes into effect on "January 1, 2027," the Verification Agent must find that exact string in the source text. If it cannot, it rejects the extraction and forces the Extraction Agent to retry with a different approach.
- 4

Enrichment & Gap Filling
Triggers external API tools

If the verified data is incomplete (e.g., a document references "Directive (EU) 2023/2413" but lacks the publication date), the Research Agent uses web search tools or legal APIs to retrieve the missing metadata and appends it to the JSON package.
- 5

Deduplication & Database Commit
Requires database read/write access

The Reconciliation Agent compares the finalized JSON against your existing single-source-of-truth database. It uses semantic similarity to detect if a document is already logged under a slightly different name. If it is unique, it writes the record; if it conflicts, it flags it for human review.

Design Considerations for Legal Data

- 1. **Human-in-the-Loop (HITL) Escalation:** No AI pipeline is 100% autonomous on day one. You must design an "escalation queue" where ambiguous documents, failed verifications, or complex deduplication conflicts are sent to you for manual review. Over time, your corrections can be used to improve the agents' instructions.
- 2. **Schema-First Design:** Before building the agents, define your target database schema explicitly. The AI needs to know exactly what constitutes a "complete" record for a piece of hydrogen legislation.
- 3. **Chain-of-Thought Reasoning:** Instruct your extraction and reconciliation agents to write out their reasoning (e.g., "I am classifying this as Czech law because it references *Zákon č. 165/2012 Sb.*") before outputting the final JSON. This makes debugging significantly easier.

Sources

- 1. [How to Build an AI Agent for Document Data Extraction - Stack AI](#)

User prompt: What are the best frameworks (like LangChain, LlamaIndex, or AutoGen) and foundational models for building an agentic legal document extraction pipeline?

Response: For a highly structured, low-tolerance legal extraction pipeline, you need a deterministic state machine rather than conversational agents. The goal is predictable, schema-validated JSON, not open-ended chat.

Here is the breakdown of the most effective frameworks and models for this architecture as of 2026.

Orchestration Frameworks

Framework	Best Used For	Verdict for Legal Extraction
LangGraph	Production-grade, stateful pipelines with complex routing, retries, and strict control flow.	Primary Choice. Unmatched for building deterministic loops (e.g., if validation fails, send back to the extractor) and managing durable state, which is essential for pausing execution so a human can review flagged conflicts.
LlamaIndex (Workflows)	Document-centric processing, RAG, and connecting to vector stores.	Co-Primary Choice. Historically a RAG tool, its new event-driven Workflows architecture is heavily optimized for ingesting, chunking, and extracting data from private documents. Many production teams use LlamaIndex for the data layer and LangGraph for the orchestration layer.
CrewAI / AutoGen (AG2)	Role-based, conversational multi-agent teams.	Skip. These excel at tasks where agents "talk" to each other to solve abstract problems. Legal extraction requires rigid data pipelines and explicit graph-based control flows, making conversational frameworks prone to latency and hallucinations.

Foundational Models

You should not use one massive model for everything. Assign models based on the specific agent's task to optimize for accuracy, context limits, and cost.

- **Parser & Extraction Agents (Vision and Schema Mastery):**
 - **Anthropic Claude 3.5 / 3.7 Sonnet:** The industry standard for adhering to strict JSON schemas, understanding nested legal definitions, and minimizing hallucinations during structured extraction.
 - **Google Gemini 1.5 Pro / 2.0 Pro:** The best choice for massive legislative texts. Its massive context window handles 100+ page EU directives natively. Its native multimodal capabilities excel at "looking" at a PDF visually to understand complex legal tables before converting them to data.
- **Reconciliation & Verification Agents (Complex Reasoning):**
 - **OpenAI o1 / o3:** These reasoning models are best utilized for complex logic checks—such as determining if a newly parsed Czech transposition law is a duplicate of an existing EU directive already in your database.
- **Privacy-First & Local Deployments:**
 - **Llama 3.3 70B / DeepSeek V3:** If your legal documents require strict data locality and cannot be sent to commercial APIs, these open-weight models can be self-hosted and offer near frontier-level performance for extraction tasks.

The Crucial Sub-Layer: Document Parsing

LLMs perform poorly if you feed them scrambled text. Before the Extraction Agent sees the document, the raw PDFs (especially multi-column EU journals or scanned Czech Sbírka documents) must be converted into clean Markdown.

Do not rely on traditional OCR. Use modern layout-aware parsers:

- **LlamaParse:** A specialized service that uses vision models to perfectly reconstruct complex document layouts, nested bullet points, and tables into Markdown.
- **Docling / Marker:** The premier open-source alternatives for converting messy PDFs into structured Markdown while preserving the exact reading order of the legal text.

User prompt: How should I design the JSON schema for extracting hydrogen-related legislation to ensure the agents capture all the right metadata?

Response: A robust JSON schema for LLM extraction does more than just define data types; it acts as the primary instruction manual for the Extraction Agent. Every key must have a highly specific description, and the schema must force the LLM to extract exact quotes to enable the Verification Agent to do its job.

Here is the schema architecture optimized for European and Czech hydrogen legislation.

The Extraction Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "document_metadata": {
      "type": "object",
      "properties": {
        "title_original": {
          "type": "string",
          "description": "The exact original title of the legislation in its native language (e.g., Czech or English)"
        },
        "jurisdiction": {
          "type": "string",
          "enum": ["EU", "CZ"],
          "description": "The governing body issuing the legislation."
        },
        "legislation_type": {
          "type": "string",
          "enum": ["Directive", "Regulation", "Act/Zákon", "Decree/Vyhláška", "Strategy", "Other"],
          "description": "The legal classification of the document."
        },
        "reference_number": {
          "type": "string",
          "description": "The CELEX number for EU documents, or the Sbírka number (e.g., 165/2012 Sb.) for Czech doc"
        },
        "dates": {
          "type": "object",
          "properties": {
            "publication_date": { "type": "string", "format": "date", "description": "Format YYYY-MM-DD." },
            "effective_date": { "type": "string", "format": "date", "description": "Format YYYY-MM-DD. When the law"
          }
        }
      }
    },
    "required": ["title_original", "jurisdiction", "legislation_type"],
    "hydrogen_specifics": {
      "type": "object",
      "properties": {
        "focus_areas": {
          "type": "array",
          "items": {
            "type": "string",
            "enum": ["Production", "Transport & Pipelines", "Storage", "Mobility/End-Use", "Certification/Guarantees"]
          },
          "description": "Which aspects of the hydrogen economy this document regulates."
        },
        "hydrogen_definitions": {
          "type": "array",
          "items": {
            "type": "string",
            "enum": ["Renewable/Green (RFNBO)", "Low-Carbon/Blue", "Nuclear/Pink", "Fossil/Grey", "Not Specified"]
          },
          "description": "The specific classifications of hydrogen mentioned or regulated in the text."
        },
        "key_obligations": {
          "type": "array",
          "items": { "type": "string" },
          "description": "A 1-2 sentence summary of the practical obligations, targets, or quotas this legislation i"
        }
      },
      "required": ["focus_areas", "hydrogen_definitions"]
    },
    "legal_relationships": {
      "type": "object",
      "properties": {
        "transposes_eu_directive": {
          "type": "string",
          "description": "If this is a Czech law, what EU Directive is it implementing? Provide the EU Directive num"
        }
      }
    }
  }
}
```

```

    "amends_legislation": {
        "type": "array",
        "items": { "type": "string" },
        "description": "List of reference numbers of previous laws this document explicitly amends or repeals."
    }
},
"verification_evidence": {
    "type": "array",
    "items": {
        "type": "string"
    },
    "description": "CRITICAL: Extract exact, verbatim quotes from the original text that justify the data extracted"
},
"required": ["document_metadata", "hydrogen_specifics", "verification_evidence"]
}

```

Schema Design Principles for Agents

1. The "Verification Evidence" Field is Mandatory LLMs are prone to hallucinating dates and relationships. By forcing the Extraction Agent to output a `verification_evidence` array containing verbatim quotes from the document, you give the Verification Agent the exact data it needs to perform a string-matching check against the original parsed Markdown. If the exact quote isn't in the source, the extraction is rejected.

2. Enums Prevent Database Fragmentation If you allow free-text for fields like `focus_areas`, you will end up with "Green Hydrogen", "Renewable H2", and "RFNBO" scattered across your database. Using strict enum arrays forces the LLM to map the document's terminology to your standardized database ontology.

3. Nullability is a Feature Legal texts are often incomplete (e.g., an amendment might not have a distinct effective date yet). You must explicitly instruct the LLM via the descriptions that it is acceptable to output `null` rather than guessing or inferring a value from outside the document's text.

4. Descriptions act as Prompts When using structured outputs (via Pydantic in Python or Zod in TypeScript), the `description` fields are passed directly to the LLM. You do not need a massive system prompt; the schema *is* the prompt. Writing clear instructions inside the schema descriptions yields much higher accuracy than external prompting.

User prompt: How do I implement this JSON schema using Python and Pydantic so I can use it with LangGraph and OpenAI/Anthropic?

Response: Implementing this schema using **Pydantic v2** and **LangChain** is straightforward. Pydantic allows you to translate the strict JSON schema into Python classes where the `Field(description="...")` parameter acts as the instruction for the LLM.

When you pass this Pydantic model to an OpenAI or Anthropic model using LangChain's `.with_structured_output()` method, LangChain automatically compiles your Pydantic classes into the native tool-calling/structured output format expected by the provider API.

Here is the complete Python code to set this up.

1. Define the Pydantic Schema

```

from typing import List, Optional
from datetime import date
from enum import Enum
from pydantic import BaseModel, Field

# -----
# 1. Enums (Restricts the LLM to predefined categories)
# -----
class Jurisdiction(str, Enum):
    EU = "EU"
    CZ = "CZ"

class LegislationType(str, Enum):
    DIRECTIVE = "Directive"
    REGULATION = "Regulation"
    ACT = "Act/Zákon"
    DECREE = "Decree/Vyhláška"
    STRATEGY = "Strategy"
    OTHER = "Other"

class FocusArea(str, Enum):
    PRODUCTION = "Production"
    TRANSPORT = "Transport & Pipelines"
    STORAGE = "Storage"
    MOBILITY = "Mobility/End-Use"
    CERTIFICATION = "Certification/Guarantees of Origin"
    SUBSIDIES = "Subsidies/State Aid"

class HydrogenDefinition(str, Enum):
    GREEN = "Renewable/Green (RFNBO)"

```

```

BLUE = "Low-Carbon/Blue"
PINK = "Nuclear/Pink"
GREY = "Fossil/Grey"
NOT_SPECIFIED = "Not Specified"

# -----
# 2. Sub-Models (For nested JSON structures)
# -----
class Dates(BaseModel):
    publication_date: Optional[date] = Field(
        default=None,
        description="Format YYYY-MM-DD. Leave null if not found."
    )
    effective_date: Optional[date] = Field(
        default=None,
        description="Format YYYY-MM-DD. When the law becomes legally binding."
    )

class DocumentMetadata(BaseModel):
    title_original: str = Field(
        description="The exact original title of the legislation in its native language."
    )
    jurisdiction: Jurisdiction = Field(
        description="The governing body issuing the legislation."
    )
    legislation_type: LegislationType = Field(
        description="The legal classification of the document."
    )
    reference_number: Optional[str] = Field(
        default=None,
        description="The CELEX number for EU documents, or the Sbírka number (e.g., 165/2012 Sb.). Essential for ded
    )
    dates: Dates = Field(default_factory=Dates)

class HydrogenSpecifics(BaseModel):
    focus_areas: List[FocusArea] = Field(
        description="Which aspects of the hydrogen economy this document regulates."
    )
    hydrogen_definitions: List[HydrogenDefinition] = Field(
        description="The specific classifications of hydrogen mentioned or regulated in the text."
    )
    key_obligations: List[str] = Field(
        description="1-2 sentence summaries of the practical obligations, targets, or quotas this legislation impose
    )

class LegalRelationships(BaseModel):
    transposes_eu_directive: Optional[str] = Field(
        default=None,
        description="If this is a Czech law, what EU Directive is it implementing? Provide the EU Directive number."
    )
    amends_legislation: List[str] = Field(
        default_factory=list,
        description="List of reference numbers of previous laws this document explicitly amends or repeals."
    )

# -----
# 3. The Main Output Model
# -----
class HydrogenLegislation(BaseModel):
    """Extraction schema for European and Czech hydrogen legislation."""
    document_metadata: DocumentMetadata
    hydrogen_specifics: HydrogenSpecifics
    legal_relationships: LegalRelationships = Field(default_factory=LegalRelationships)
    verification_evidence: List[str] = Field(
        description="CRITICAL: Extract exact, verbatim quotes from the original text that justify the data extracted
    )

```

2. Connect to the LLM (OpenAI or Anthropic)

Now you bind the Pydantic model to your chosen foundation model.

```

import os
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic

# Set up your environment variables
# os.environ["OPENAI_API_KEY"] = "your-openai-key"
# os.environ["ANTHROPIC_API_KEY"] = "your-anthropic-key"

```

```
# 1. Initialize the base model (you can swap these interchangeably)
# Using OpenAI (recommended for o1/o3 reasoning or gpt-4o data extraction)
llm = ChatOpenAI(model="gpt-4o", temperature=0)

# OR using Anthropic (excellent at adhering to strict JSON schemas)
# llm = ChatAnthropic(model="claude-3-5-sonnet-latest", temperature=0)

# 2. Bind the Pydantic schema to the model
# The strict=True flag (for OpenAI) enforces exact schema compliance
structured_llm = llm.with_structured_output(HydrogenLegislation, strict=True)

# 3. Create your Extraction Agent logic
def extract_legislation(markdown_text: str) -> HydrogenLegislation:
    system_prompt = """
    You are an expert legal data extraction agent for the Czech Republic and European Union.
    Analyze the following legislative text and extract the required fields exactly as defined in the schema.
    If a value is not explicitly stated in the text, return null. Do not hallucinate external knowledge.
    """

    # Send the prompt and the parsed text to the model
    result = structured_llm.invoke(f"{system_prompt}\n\n{markdown_text}")
    return result

# --- Example Usage ---
dummy_markdown = """
Zákon č. 165/2012 Sb., o podporovaných zdrojích energie.
Tento zákon transponuje směrnici Evropského parlamentu a Rady (EU) 2018/2001...
Účinnost od: 1. ledna 2013. Zákon definuje zelený vodík jako plyn vyrobený z obnovitelných zdrojů...
"""

# The output is a fully validated Python object, not a raw string
extracted_data = extract_legislation(dummy_markdown)

print(extracted_data.document_metadata.reference_number)
# Output: "165/2012 Sb."

print(extracted_data.model_dump_json(indent=2))
# Output: A perfect JSON string ready to be inserted into your database
```

Why this setup is powerful in LangGraph

Because `.with_structured_output()` returns a fully typed Pydantic object (not a string you have to manually parse with `json.loads`), you can use the output directly in LangGraph's state routing.

For example, in a LangGraph workflow, you can check if `extracted_data.verification_evidence` is empty. If it is, the graph can automatically route the text back to the LLM with an error prompt ("You failed to provide evidence quotes, try again") without breaking the application.

Sources

1. [7 LangChain Structured-Output Tricks | by Thinking Loop - Medium](#)
2. [LangChain Structured Outputs: A Guide to Tools and Methods](#)

User prompt: How do I write the Python logic for the Verification Agent to check if the exact quotes in 'verification_evidence' actually exist in the original text?

Response: Because documents converted from PDFs and web pages often contain unpredictable line breaks, hyphenation, and spacing, a simple quote in text check is too brittle. The Verification Agent must normalize both the original text and the extracted quotes before comparing them.

The Verification Agent Logic

This Python function uses string normalization and regular expressions to strip out formatting discrepancies, ensuring the LLM isn't penalized for a line break that existed in the source PDF but was omitted in the JSON output.

```
import re
import unicodedata
from typing import Dict, List

def normalize_text(text: str) -> str:
    """
    Standardizes text to prevent false-negative verification failures
    caused by PDF parsing artifacts.
    """
    if not text:
        return ""

    # Convert to lowercase
```

```

text = text.lower()

# Normalize unicode characters (handles smart quotes, distinct space characters)
text = unicodedata.normalize('NFKD', text)

# Replace all whitespace variations (newlines, tabs, multiple spaces) with a single space
text = re.sub(r'\s+', ' ', text)

# Optional: Remove punctuation if you want even looser matching,
# but for legal texts, keeping punctuation is usually safer.

return text.strip()

def verify_evidence(original_markdown: str, extracted_quotes: List[str]) -> Dict:
    """
    Cross-checks the LLM's extracted evidence against the source document.
    Returns a dictionary with the validation status and any failing quotes.
    """
    if not extracted_quotes:
        return {
            "is_valid": False,
            "error": "No verification evidence provided by the extractor."
        }

    normalized_source = normalize_text(original_markdown)
    failed_quotes = []

    for quote in extracted_quotes:
        normalized_quote = normalize_text(quote)

        # Check if the exact (normalized) string exists in the source
        if normalized_quote not in normalized_source:
            failed_quotes.append(quote)

    return {
        "is_valid": len(failed_quotes) == 0,
        "failed_quotes": failed_quotes
    }

```

Implementing the Retry Loop

When integrated into your pipeline, the Verification Agent does not just print an error—it dynamically feeds the failure back to the Extraction Agent.

If `verify_evidence` returns `is_valid: False`, your orchestration framework (like LangGraph) should intercept the `failed_quotes` list and generate a targeted correction prompt, such as:

"You extracted the following quotes as evidence, but they do not exist verbatim in the source text: '[Failed Quote 1]', '[Failed Quote 2]'. Please review the document and extract the exact phrasing without altering any words, or return null if the data is not actually present."

This deterministic feedback loop forces the LLM to self-correct its hallucinations before the data ever reaches your final hydrogen legislation database.

User prompt: How do I wire this verification function and the extraction model together using a LangGraph state graph to automate the retry loop?

Response: To wire these components into a self-correcting loop, you use LangGraph's `StateGraph`. The framework passes a central "State" dictionary between the nodes. The Extractor writes the JSON to the state, the Verifier reads it and checks for errors, and a conditional edge decides whether to end the process or loop back.

1. Define the State and Nodes

```

from typing import List, Optional, TypedDict
from langgraph.graph import StateGraph, START, END
from langchain_core.messages import HumanMessage, SystemMessage

# 1. Define the shared State for the graph
class PipelineState(TypedDict):
    original_text: str
    extracted_data: Optional[HydrogenLegislation] # From our Pydantic schema
    errors: List[str]
    attempts: int

# 2. Node: Extraction Agent
def extraction_node(state: PipelineState):
    text = state["original_text"]
    errors = state.get("errors", [])

```

```

    attempts = state.get("attempts", 0)

    # Base instructions
    system_prompt = """
    You are an expert legal extraction agent. Extract the required fields from the
    following hydrogen legislation text exactly as defined in the schema.
    """

    # If this is a retry, append the strict correction instructions
    if errors:
        system_prompt += "\n\nCRITICAL ERROR: In your previous attempt, you hallucinated evidence."
        system_prompt += "\nThe following quotes were NOT found in the source text:\n"
        system_prompt += "\n".join(f"- {err}" for err in errors)
        system_prompt += "\n\nFix this. Only extract EXACT verbatim quotes."

    messages = [
        SystemMessage(content=system_prompt),
        HumanMessage(content=text)
    ]

    # structured_llm is the Pydantic-bound LLM from the previous step
    result = structured_llm.invoke(messages)

    return {"extracted_data": result, "attempts": attempts + 1}

# 3. Node: Verification Agent
def verification_node(state: PipelineState):
    extracted = state["extracted_data"]
    original = state["original_text"]

    # Run the normalization and verification logic we wrote earlier
    verification = verify_evidence(original, extracted.verification_evidence)

    if not verification["is_valid"]:
        # Pass the specific failed quotes back into the state's error list
        return {"errors": verification["failed_quotes"]}

    # If valid, clear any previous errors
    return {"errors": []}

```

2. Define the Routing Logic

You need a function that looks at the state after verification and decides what to do next. It prevents infinite loops by capping the retries.

```

def route_after_verification(state: PipelineState):
    if not state["errors"]:
        return "success"

    if state["attempts"] >= 3: # Max retries reached
        return "escalate"

    return "retry"

```

3. Build and Compile the Graph

Now, attach the nodes and the router together into an executable graph.

```

# Initialize the graph with our custom State definition
workflow = StateGraph(PipelineState)

# Add the nodes
workflow.add_node("extractor", extraction_node)
workflow.add_node("verifier", verification_node)

# Define the flow
workflow.add_edge(START, "extractor")
workflow.add_edge("extractor", "verifier")

# Add the conditional routing logic
workflow.add_conditional_edges(
    "verifier",
    route_after_verification,
    {
        "success": END,                # Data is verified, exit graph
        "retry": "extractor",          # Send back to LLM with error context
        "escalate": END                 # In production, route to a Human Review node
    }
)

```



```
# Compile into a runnable application
app = workflow.compile()
```

4. Run the Pipeline

To execute the pipeline, pass your parsed document into the initial state. The graph will autonomously cycle through extraction and verification until it succeeds or hits the retry limit.

```
initial_state = {
    "original_text": dummy_markdown, # Your parsed PDF text
    "extracted_data": None,
    "errors": [],
    "attempts": 0
}

# Stream the execution to watch the agents work
for step in app.stream(initial_state):
    for node_name, state_update in step.items():
        print(f"--- Finished running: {node_name} ---")
        if "errors" in state_update and state_update["errors"]:
            print(f"Failed verification. Retrying...")

# Access the final, verified JSON data
final_state = app.get_state(config).values
if not final_state["errors"]:
    print(final_state["extracted_data"].model_dump_json(indent=2))
```

User prompt: How do I add a Reconciliation Agent to this graph to check if the extracted legislation already exists in my database?

Response: To add the Reconciliation Agent, we will append a new node to the end of your graph. This agent will activate only after the Verification Agent has successfully validated the extraction.

In legal databases, deduplication should use a two-step approach:

1. **Deterministic Matching (Primary):** Check if the `reference_number` (e.g., CELEX number or Sbírka number) already exists in your database. This is 100% accurate.
2. **Semantic/Fuzzy Matching (Fallback):** If the reference number is missing or inconsistently formatted (e.g., "RED II" vs "Directive (EU) 2018/2001"), the agent converts the `title_original` into a vector embedding and checks for high semantic similarity in your database.

Here is how to update your LangGraph code to include this agent.

1. Update the State

First, we need to add a field to our `PipelineState` to track the deduplication result.

```
class PipelineState(TypedDict):
    original_text: str
    extracted_data: Optional[HydrogenLegislation]
    errors: List[str]
    attempts: int
    duplicate_of: Optional[str] # NEW: Will hold the ID of an existing DB record if found
    action_taken: Optional[str] # NEW: "inserted", "merged", or "flagged_for_review"
```

2. Create the Reconciliation Logic

We will simulate a database connection. In reality, you would connect this to a SQL database (for exact matches) and a Vector Database like Pinecone, Milvus, or pgvector (for semantic matches).

```
# Simulated database query functions
def check_db_exact_match(ref_number: str, jurisdiction: str) -> Optional[str]:
    """Queries SQL database for exact reference number match."""
    # Example: SELECT id FROM legislation WHERE ref_number = ? AND jurisdiction = ?
    existing_db = {"165/2012 Sb.": "db_record_uuid_994"}
    return existing_db.get(ref_number)

def check_db_semantic_match(title: str) -> Optional[str]:
    """Queries Vector DB for highly similar titles (> 95% similarity)."""
    # Example: Returns database ID if a nearly identical title exists
    return None

# 3. Node: Reconciliation Agent
def reconciliation_node(state: PipelineState):
    data = state["extracted_data"]

    # Step 1: Deterministic Check (Fast and cheap)
    if data.document_metadata.reference_number:
        match_id = check_db_exact_match(
            data.document_metadata.reference_number,
```

```

        data.document_metadata.jurisdiction
    )
    if match_id:
        return {"duplicate_of": match_id, "action_taken": "flagged_for_review"}

# Step 2: Semantic Check (Uses Embeddings)
# If no reference number, or exact match failed, check title semantics
match_id = check_db_semantic_match(data.document_metadata.title_original)
if match_id:
    return {"duplicate_of": match_id, "action_taken": "flagged_for_review"}

# Step 3: No duplicates found, safe to insert
# db.insert(data.model_dump())
return {"duplicate_of": None, "action_taken": "inserted"}

```

3. Update the Graph Routing

We must change the graph so that a successful verification no longer goes to END, but instead routes to our new reconciliation_node.

```

def route_after_verification(state: PipelineState):
    if not state["errors"]:
        return "reconcile" # CHANGED: Now routes to reconciliation instead of END

    if state["attempts"] >= 3:
        return "escalate"

    return "retry"

# --- Rebuilding the Graph ---
workflow = StateGraph(PipelineState)

workflow.add_node("extractor", extraction_node)
workflow.add_node("verifier", verification_node)
workflow.add_node("reconciler", reconciliation_node) # NEW NODE

workflow.add_edge(START, "extractor")
workflow.add_edge("extractor", "verifier")

# Updated conditional edges from verifier
workflow.add_conditional_edges(
    "verifier",
    route_after_verification,
    {
        "reconcile": "reconciler", # Routes to deduplication
        "retry": "extractor",
        "escalate": END
    }
)

# The reconciler is the final step, so it goes to END
workflow.add_edge("reconciler", END)

app = workflow.compile()

```

Handling Deduplication Conflicts (Human-in-the-Loop)

Notice that if the reconciliation_node finds a duplicate, it sets action_taken to "flagged_for_review" rather than aggressively overwriting your database.

Because legislative updates are complex (e.g., an amendment might have the exact same title as the original law but a different date), it is risky for an AI to blindly merge records. Setting a "flagged" status allows the pipeline to save the JSON to a staging table in your database, where you can manually review the conflict in your UI before clicking "Merge" or "Create New Version."

User prompt: How do I add a Research Agent using tool-calling to automatically fetch missing metadata (like publication dates) from EUR-Lex or e-Sbirka before it gets saved to the database?

Response: Integrating a Research Agent turns your pipeline from a passive parser into an active enrichment engine. Instead of merely extracting what is in the document, this agent can recognize when critical metadata is missing (e.g., the extraction returned null for publication_date) and automatically query external APIs to fill the gap.

To do this in LangGraph, we will introduce a new node that uses **Tool Calling**.

Here is how to design and integrate the Research Agent.

1. Define the Tools

A tool is simply a Python function wrapped with a LangChain decorator (@tool). The decorator allows the LLM to understand what the function does and what arguments it requires.

We will create two placeholder tools for the EUR-Lex and e-Sbírka APIs.

```
from langchain_core.tools import tool
import json

@tool
def search_eurlex_metadata(celex_number: str) -> str:
    """
    Fetches missing metadata (like publication date) for an EU Directive or Regulation
    from the EUR-Lex database using its CELEX number.
    """
    # In reality, you would make an HTTP request to the EUR-Lex SPARQL endpoint here.
    # For this example, we mock the response.
    mock_db = {
        "32018L2001": {"publication_date": "2018-12-21"}, # RED II
        "32023L2413": {"publication_date": "2023-10-31"} # RED III
    }

    result = mock_db.get(celex_number)
    if result:
        return json.dumps(result)
    return "Metadata not found in EUR-Lex."

@tool
def search_esbirka_metadata(sbirka_number: str) -> str:
    """
    Fetches missing metadata (like publication date or effective date) for Czech legislation
    from the e-Sbírka database using its Sbírka number (e.g., '165/2012 Sb.').
    """
    # Mocking a request to the Czech e-Sbírka API
    mock_db = {
        "165/2012 Sb.": {"publication_date": "2012-05-30"}
    }

    result = mock_db.get(sbirka_number)
    if result:
        return json.dumps(result)
    return "Metadata not found in e-Sbírka."

# Group the tools together
tools = [search_eurlex_metadata, search_esbirka_metadata]
```

2. Create the Research Node

The Research Agent needs to be smart enough to know *when* to use a tool. We achieve this by binding the tools to the LLM.

When the LLM decides it needs information, it will pause its text generation, output a structured tool call request, wait for your Python code to execute the function and return the result, and *then* formulate its final answer.

```
from langchain_core.messages import ToolMessage
from langgraph.prebuilt import ToolNode

# 1. Bind the tools to a capable model (like GPT-4o or Claude 3.5 Sonnet)
# We don't use the structured output model here because we want it to use tools
research_llm = llm.bind_tools(tools)

# 2. Define the Research Node
def research_node(state: PipelineState):
    data = state["extracted_data"]

    # Check if we are missing critical dates that we need to research
    missing_pub_date = data.document_metadata.dates.publication_date is None

    # If we have the reference number but are missing the date, trigger the agent
    if missing_pub_date and data.document_metadata.reference_number:
        ref_num = data.document_metadata.reference_number
        jurisdiction = data.document_metadata.jurisdiction

        system_prompt = f"""
        You are a legal research assistant. The extracted data is missing its publication date.
        Use the available tools to search the appropriate database for reference number '{ref_num}'.
        If the jurisdiction is EU, use the EUR-Lex tool. If it is CZ, use the e-Sbírka tool.
        Return ONLY the missing date in YYYY-MM-DD format, or 'NOT_FOUND'.
        """

        # Invoke the LLM. If it decides to use a tool, it will return a message with tool_calls.
        messages = [SystemMessage(content=system_prompt)]
        response = research_llm.invoke(messages)

    # We need a mini-loop inside this node to actually execute the tool
```

```

if hasattr(response, 'tool_calls') and len(response.tool_calls) > 0:
    # A simpler way is to use LangGraph's prebuilt ToolNode, but for clarity
    # we will execute it directly here.
    for tool_call in response.tool_calls:
        # Find the matching tool function
        tool_func = next((t for t in tools if t.name == tool_call['name']), None)
        if tool_func:
            # Execute the python function with the arguments provided by the LLM
            tool_result = tool_func.invoke(tool_call['args'])

            # Append the result back to the message history and call the LLM again
            messages.append(response) # The LLM's request to use the tool
            messages.append(ToolMessage(
                content=str(tool_result),
                tool_call_id=tool_call['id']
            ))

        # The LLM reads the tool output and gives its final answer
        final_response = research_llm.invoke(messages)

    # If the LLM successfully found the date, parse it and update our state
    if "NOT_FOUND" not in final_response.content:
        # In a production app, you'd want to strictly parse this date
        # For this example, we just update the Pydantic object
        try:
            from datetime import datetime
            parsed_date = datetime.strptime(final_response.content.strip(), "%Y-%m-%d").date()
            data.document_metadata.dates.publication_date = parsed_date
            print(f"Research Agent found missing date: {parsed_date}")
        except ValueError:
            pass # Failed to parse date string

    # Return the enriched data back into the state
    return {"extracted_data": data}

```

3. Wire It Into the Graph

The Research Agent should run *after* Verification (so we know the core extraction is accurate) and *before* Reconciliation (so the enriched metadata can be used to help identify duplicates).

Update your graph wiring like this:

```

# Add the new node
workflow.add_node("researcher", research_node)

# Update the conditional routing from the Verifier
# Now, if verification succeeds, it goes to the researcher first
workflow.add_conditional_edges(
    "verifier",
    route_after_verification,
    {
        "success": "researcher", # CHANGED: Was "reconcile" in the previous step
        "retry": "extractor",
        "escalate": END
    }
)

# Connect the researcher to the reconciler
workflow.add_edge("researcher", "reconciler")

```

Why this design is effective

By isolating tool-calling to its own node, you keep the initial extraction step fast and deterministic. The LLM during extraction doesn't have the cognitive burden of deciding whether to use a tool; it just extracts what is on the page. The Research Agent only activates—and incurs API costs—when it is strictly necessary to fill a gap in your schema.